FEU ALABANG

COMPUTER SCIENCE DEPARTMENT

(CS0065 - CS ELECTIVE- INTELLIGENT SYSTEMS)

MIDTERM EXAMINATION
PROJECT

<Añonuevo, Kassandra Margarette F.>
20231000026

<Cuesta, Gabriel A.>
20231000483

<Hernaez, Angelee Louise R.>
20231000306

AN43
October 14, 2026

Critical-thinking questions:

1. Which criterion or rule should receive the highest priority?

- Since there is no official priority scoring system established for our campus service queue, we created our own scoring rules based on what we believe should be prioritized. Urgency should receive the highest priority. However, the program also considers student type, special assistance, document completeness, and estimated duration, while arrival time is used as a tie-breaker to maintain fairness.

2. What should happen when no option satisfies every requirement?

- The program should still rank the requests based on their total priority scores, even if some requirements are not fully satisfied. Missing values are handled using the default values defined in the program.

3. How should ties, conflicts, or human overrides be handled and explained?

- When requests have equal priority scores, the program uses arrival time and then record ID to determine their order.

Interpretation: *Write a short explanation of the results and the project's limitations.*

Priority Scoring produced a different order from FIFO because it considers urgency, student type, special assistance, document completeness, and service duration. FIFO only considers arrival time. Therefore, Priority Scoring can place urgent or special cases earlier in the queue.

The weights were manually assigned and may not represent every campus situation. The results also depend on accurate input data. The system should support staff decisions rather than completely replace human judgment.

Instructions:

GROUP 8

Campus Service Queue Prioritization

Decision Support / Scheduling

Scenario: A campus service office wants to organize its queue using arrival time, urgency, service type, and expected service duration.

System question: How should the available records be ranked or organized to produce: Recommended service order?

Implementation cope	Use priority scoring with simple queue rules
Primary algorithm	Priority Scoring
Compare with	First-In First-Out
System input	Arrival time, service type, urgency, estimated service duration
System output	Recommended service order
Python method	import pandas as pd; import numpy as np

Part 1 Understand the Dataset

Class activity: Identify reference-only fields, analytical inputs, any required target label, the generated output, missing information, and one privacy, fairness, safety, or accessibility concern.

Part 2 Prepare and Check the Dataset

Minimum requirement: Create approximately 30 to 100 records using four to eight relevant input features. Validate ranges and categories, document every rule or weight, retain identifiers for reporting, and define a transparent tie-breaking rule.

Part 3 Build the KNIME Workflow

Suggested workflow: CSV Reader -> Column Filter -> Missing Value -> Rule Engine -> Math Formula -> Normalizer -> Sorter -> Top k

Evaluation: ranking consistency, sensitivity to rules or weights, validity, and explainability

Part 4 Recreate the Process in Python

Required process: Use import pandas as pd; import numpy as np. Load the CSV with pandas, apply the documented rules or weights, calculate scores, sort the records, handle ties, and export the ranked results.

Part 5 Create the Streamlit Interface

Interface requirement: Collect or upload these inputs: Arrival time, service type, urgency, estimated service duration. Validate incomplete or invalid values, run the same Python process, and display recommended service order with a brief explanation.

Part 6 Test Explain and Reflect

Testing: Test normal, missing, boundary, and uncertain cases. Compare the primary and comparison algorithms using one results table or appropriate chart.

Critical-thinking questions:

1. Which criterion or rule should receive the highest priority?
2. What should happen when no option satisfies every requirement?
3. How should ties, conflicts, or human overrides be handled and explained?

Interpretation: Write a short explanation of the results and the project's limitations.

Required Outputs

30-100 record dataset | Data dictionary | KNIME workflow | Python implementation | Streamlit interface | Comparison table or chart | Short interpretation and limitations